

Algoritmi e Strutture Dati–parte I

lez. 15

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

BFS(V, E)

```
for ogni vertice  $u \in V[G] - \{s\}$ 
    color[ $u$ ]  $\leftarrow$  WHITE
     $\pi[u] \leftarrow$  NIL
     $d[u] \leftarrow \infty$ 
color[ $s$ ]  $\leftarrow$  GRAY
 $d[s] \leftarrow 0$ 
 $\pi[s] \leftarrow$  NIL
 $Q \leftarrow \emptyset$ 
ENQUEUE( $Q, s$ );
while ( $Q \neq \emptyset$ )
    do  $u \leftarrow$  DEQUEUE( $Q$ )
        for ogni  $v \in Adj[u]$ 
            do if color[ $v$ ] = WHITE
                then color[ $v$ ]  $\leftarrow$  GRAY
                     $\pi[v] \leftarrow u$ 
                     $d[v] \leftarrow d[u] + 1$ 
                    ENQUEUE( $Q, v$ );
color[ $u$ ]  $\leftarrow$  BLACK;
```

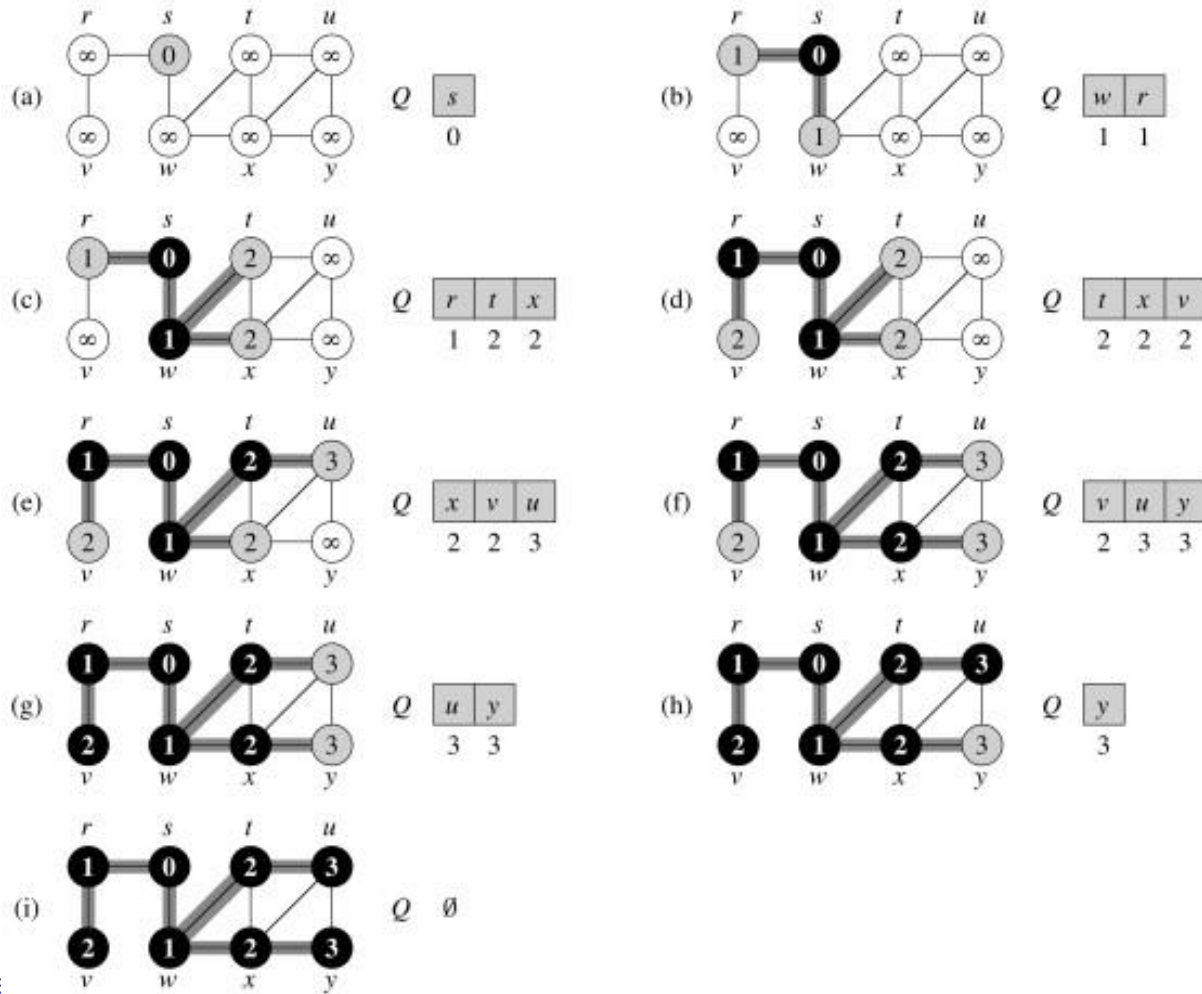
BFS

- Con l'eccezione del vertice sorgente s , le righe 1-4 colorano tutti i vertici in bianco, assegnano alla variabile $d[u] \leftarrow \infty$ e $\pi[u] \leftarrow \text{NULL}$ per ogni vertice u
- Le righe 6-9 inizializzano $d[s] \leftarrow 0$, assegnano il predecessore della sorgente a NULL ed inizializzano la coda Q in modo che contenga solo la sorgente s .
- Il ciclo while (righe 10-18) si ripete finché restano vertici grigi

(cont.)

- Il ciclo for (linee 12-17) esamina ciascun vertice della lista di adiacenza di u . Se è bianco lo colora di grigio, la sua distanza è impostata a $d[u] + 1$, il suo predecessore diventa u , ed infine viene accodato alla coda Q .
- Dopo che **tutti i nodi della sua lista di adiacenza sono stati esaminati, u diventa NERO.**

BFS



Complessità Computazionale della BFS

- Le operazioni di inserimento e di cancellazione di un elemento dalla coda è $O(1)$, pertanto la coda in totale richiede $O(V)$.
- La lista di adiacenza di ogni vertice è esaminata al più una volta, quando questo viene eliminato dalla coda. Il tempo totale per esaminare tutte le liste di adiacenza è pari alla loro somma, ossia $\Theta(E)$.
- Il tempo di esecuzione totale è $O(V + E)$

Cammino Minimo

- Si definisce la **distanza di cammino minimo** $\delta(s, v)$ da s a v come il minimo numero di **archi in un cammino qualsiasi che va dal vertice s al vertice v** ; se non esiste alcun cammino che va da s a v , allora $\delta(s, v) = \infty$.
- Un **cammino di lunghezza $\delta(s, v)$ è detto cammino minimo da s a v** .

Lemma 1

- Se $G = (V, E)$ è un grafo e $s \in V$ è un vertice arbitrario, allora per qualsiasi arco $(u, v) \in E$ si ha $\delta(s, v) \leq \delta(s, u) + 1$

Dimostrazione

- Se u è un vertice raggiungibile da s , anche v lo è. Pertanto il cammino da s a v non può essere maggiore del cammino da s ad u più uno, ossia l'arco (u, v) .
- Viceversa, se u non è raggiungibile, allora $\delta(s, u) = \infty$ e quindi la disuguaglianza è valida sia nel caso che v sia raggiungibile da s , sia che non lo sia.

Lemma 2

(del limite superiore)

- Sia $G = (V, E)$ e supponiamo che *BFS* venga eseguita sul grafo G da un dato vertice sorgente $s \in V$. Al termine della procedura, per ogni vertice $v \in V$, il valore $d[v]$ soddisfa la relazione $\delta(s, v) \leq d[v]$.

Dimostrazione

- Si dimostra per induzione che $\delta(s, v) \leq d[v]$, per ogni vertice $v \in V$.
- Il caso base dell' induzione si ha quando la sorgente s è stata inserita nella coda. L' ipotesi è verificata, perché si ha $d[s] = \delta(s, s) = 0$ e $d[v] = \infty \geq \delta(s, v)$ per ogni $v \in V - \{s\}$.
- Per il passaggio induttivo, consideriamo un vertice bianco v che viene scoperto durante la visita a partire dal vertice u . L' ipotesi induttiva implica che $\delta(s, u) \leq d[u]$.

(cont.)

- Ma si ha per l' ipotesi induttiva e per il Lemma 1:
 $d[v] = d[u] + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$
- Il vertice v sarà inserito nella coda e non sarà più reinserito visto che viene colorato di grigio. Pertanto il valore di $d[v]$ non cambierà più e l' ipotesi resterà valida.

Lemma 3

- Supponiamo che durante l' esecuzione di *BFS* su un grafo $G = (V, E)$, la coda Q contenga i vertici $\langle v_1, v_2, \dots, v_r \rangle$, dove v_1 è l' inizio della coda Q e v_r è la fine. Allora valgono:
 - $d[v_r] \leq d[v_1] + 1$
 - $d[v_i] \leq d[v_{i+1}]$ per $i = 1, \dots, r - 1$

Dimostrazione

- Si dimostra per induzione.
- Inizialmente, quando la coda contiene soltanto s , il lemma è valido.
- Per il passaggio induttivo, dobbiamo provare che il lemma è valido dopo l'inserimento e la cancellazione di un vertice dalla coda. Se il vertice v_1 viene eliminato dalla coda, v_2 è la nuova testa della coda. Per l'ipotesi induttiva $d[v_r] \leq d[v_1] + 1 \leq d[v_2] + 1$.
- Il lemma è dimostrato se v_2 è l'inizio della coda.

(cont.)

- Quando inseriamo nella coda un vertice v , diventa v_{r+1} . È stato eliminato dalla coda u , la cui lista di adiacenza è attualmente presa in esame.
- Se u era l'unico elemento della coda, la sua rimozione svuota la coda, dopodichè ogni vertice v avrà lo stesso valore $d[v] = d[u] + 1$ e le disuguaglianze sono verificate.

(cont.)

- Altrimenti per l' ipotesi induttiva il nuovo inizio di coda ha $d[v_1] \geq d[u]$. Quindi
$$d[v_{r+1}] = d[v] = d[u] + 1 \leq d[v_1] + 1$$
- Per l' ipotesi induttiva si ha:
$$d[v_r] \leq d[u] + 1$$
$$d[v_r] \leq d[u] + 1 = d[v] = d[v_{r+1}]$$
- Il lemma viene dimostrato quando v è inserito nella coda.

Corollario

- Supponiamo che i vertici v_i e v_j siano inseriti nella coda durante l'esecuzione di BFS e che v_i sia inserito prima di v_j . Allora $d[v_i] \leq d[v_j]$.

Teorema della Correttezza della Visita in Ampiezza

- Sia $G = (V, E)$ un grafo e supponiamo che la procedura *BFS* venga eseguita sul grafo G da un dato vertice sorgente $s \in V$. Allora, durante la sua esecuzione, *BFS* scopre tutti i vertici $v \in V$ che sono raggiungibili dalla sorgente s , e alla fine dell'esecuzione, $d[v] = \delta(s, v)$ per ogni $v \in V$. Inoltre, per qualsiasi vertice $v \neq s$ che è raggiungibile da s , uno dei cammini minimi da s a v è un cammino minimo da s a $\pi[v]$ seguito dall'arco $(\pi[v], v)$.

Dimostrazione

- Supponiamo per assurdo che qualche vertice riceva un valore d che non sia uguale alla distanza del suo cammino minimo. Sia v il vertice con il minimo $\delta(s, v)$ che riceve questo valore errato d . Per il Lemma2 del limite superiore $\delta(s, v) \leq d[v]$ e quindi $\delta(s, v) < d[v]$. Il vertice deve essere raggiungibile altrimenti si avrebbe l'assurdo $\delta(s, v) = \infty \geq d[v]$.
- Sia u il padre di v in un cammino minimo da s a v , con $\delta(s, v) = \delta(s, u) + 1$.

(cont.)

- Mettendo insieme queste proprietà, si ha $d[v] > \delta(s, v) = \delta(s, u) + 1 = d[u] + 1$.
- Consideriamo il momento in cui *BFS* elimina il vertice v dalla coda.
- Se v è bianco allora *BFS* imposta $d[v] = d[u] + 1$ che contraddice l'affermazione precedente.
- Se v è nero e per il corollario 1 è $d[v] \leq d[u]$ che contraddice l'affermazione in blu.

(cont.)

- Se il vertice è grigio, allora è stato colorato dopo la cancellazione dalla coda di qualche vertice w che era stato cancellato prima di u e per il quale $d[v] = d[w] + 1$. Ma per il corollario 1 $d[w] \leq d[u]$ e quindi $d[v] \leq d[u] + 1$ che contraddice la disuguaglianza in blu.
- Pertanto concludiamo che $d[v] = \delta(s, v)$ per ogni $v \in V$.
- Tutti i vertici sono raggiungibili da s devono essere scoperti, altrimenti avrebbero il d infinito.

(cont.)

- Infine osserviamo che se vale $\pi[v] = u$ allora $d[v] = d[u] + 1$. Quindi si può ottenere un cammino minimo da s a v prendendo un cammino minimo da s a $\pi[v]$ e poi attraversando l'arco $(\pi[v], v)$.

Visita in Profondita` (DFS)

- La strategia seguita dalla visita in profondita` (depth-first-search) consiste nell' esplorare il grafo andando ad ogni istante il piu` possibile in profondita`.
- Nella DFS gli archi vengono scoperti a partire dall' ultimo vertice scoperto che abbia ancora vertici non scoperti uscenti da esso
- Quando tutti gli archi uscenti da v sono scoperti, la visita torna indietro per esplorare gli archi uscenti dal vertice dal quale v era stato scoperto.

DFS (cont.)

- Il processo continua fino a scoprire tutti i vertici che sono raggiungibili dal vertice sorgente originario
- Se rimane qualche vertice non ancora scoperto, viene selezionato uno di essi e diventa nuovo sorgente e la ricerca riparte da esso.
- L'intero processo viene ripetuto sino a quando tutti i vertici del grafo non sono stati scoperti.

DFS (cont.)

- Come nella *BFS* quando un vertice v viene scoperto, durante l'ispezione della lista di adiacenza di un vertice u già scoperto, viene fatta l'assegnazione $\pi[v] = u$
- A differenza della *BFS* dove il sottografo dei vertici scoperti è un albero, nel caso della *DFS* può essere una **foresta DFS**, perché la visita può essere ripetuta da più sorgenti.
- I vertici possono essere colorati di **bianco**, **grigio** e **nero**.

Sottografo dei Predecessori

- Il sottografo dei predecessori in una visita di profondità è $G_\pi = (V, E_\pi)$ dove $E_\pi = \{(\pi[v], v) : v \in V \text{ and } \pi[v] \neq NIL\}$
- Il sottografo forma una foresta **DF** composta da vari *alberi DF*.
- Gli archi sono detti **archi d' albero**.

DFS

- Inizialmente tutti i nodi sono colorati di bianco.
- I nodi sono **bianchi quando non sono stati scoperti**, **grigi quando viene scoperto**; **neri quando la visita è finita** (ossia la lista di adiacenza è stata completamente ispezionata).
- Questa tecnica assicura che ogni vertice vada a finire in un solo *albero DF* e quindi che tutti gli ***alberi DF* siano disgiunti**.
- *DFS* associa ad ogni vertice v due etichette $d[v]$ e $f[v]$; $d[v]$ indica il tempo in cui è stato scoperto; $f[v]$ quando è terminata la visita.

(cont.)

- I valori $d[v]$ e $f[v]$ sono compresi tra 1 e $2|V|$ in quanto ogni vertice può essere scoperto e visitato completamente una sola volta.
- Inoltre si ha sempre $d[v] < f[v]$.
- Il vertice è bianco prima del tempo $d[v]$; grigio quando ha un tempo compreso tra $d[v]$ e $f[v]$; e nero per un tempo superiore a $f[v]$.

DFS(G)

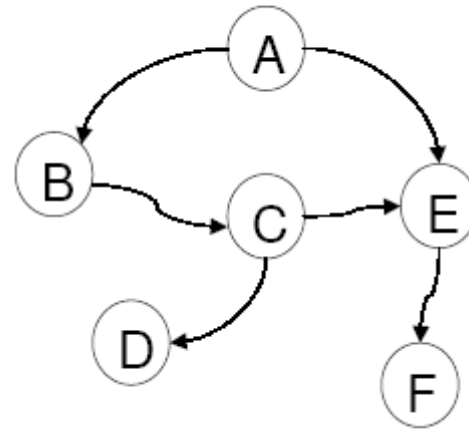
```
for each  $u \in V[G]$   
     $color[u] \leftarrow WHITE$   
     $\pi[u] \leftarrow NULL$   
 $time \leftarrow 0$   
for each  $u \in V[G]$   
    do if  $color[u] = WHITE$   
        then  $DFS\_VISIT(u)$ 
```

DFS_VISIT(u)

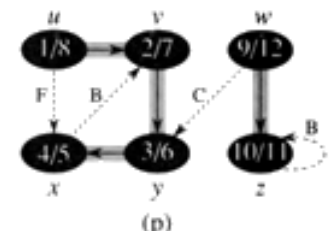
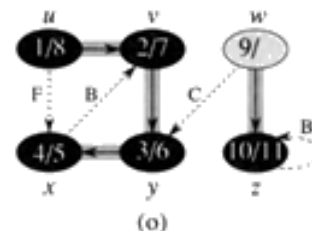
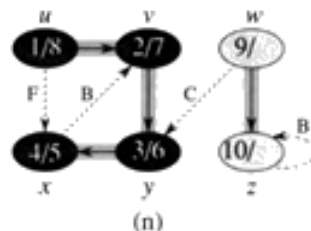
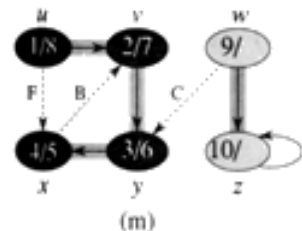
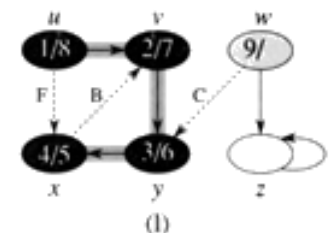
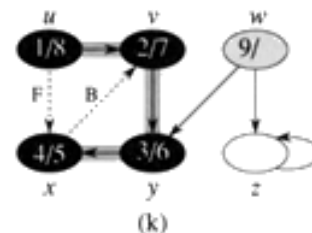
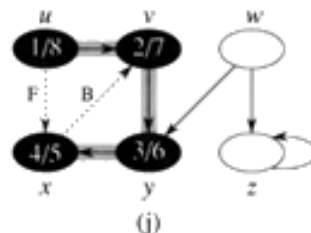
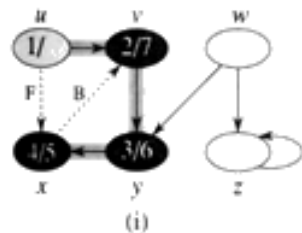
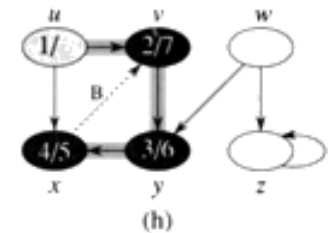
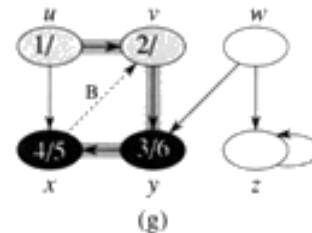
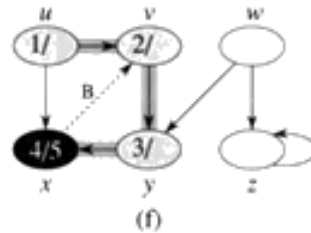
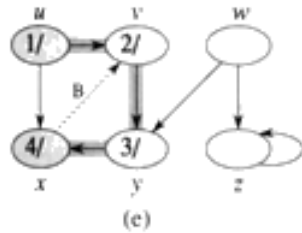
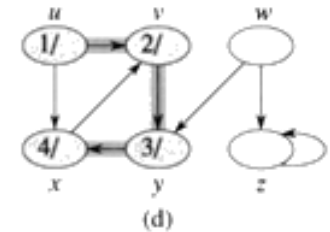
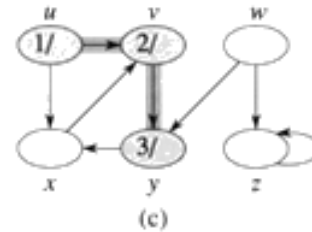
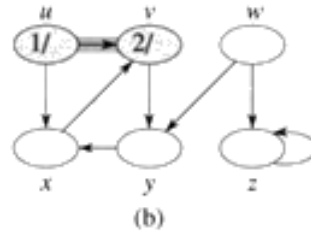
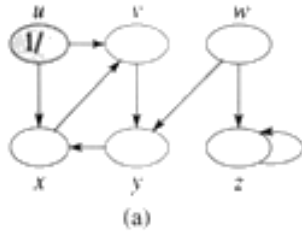
1. $time \leftarrow time + 1$
2. $d[u] \leftarrow time$
3. $color[u] \leftarrow GRAY$
4. **for** each $v \in Adj[u]$
5. **if** $color[v] = WHITE$
6. $\pi[v] = u$
7. $DFS_VISIT[v]$
8. $color[u] \leftarrow BLACK$
9. $time \leftarrow time + 1$
10. $f[u] \leftarrow time$

Risultato visita in profondità

ABCDEF



DFS



Grafi: visita in profondità

- visita_profondità(V)
 1. Passo base: se $V = \emptyset$ allora esci;
 2. Passo di induzione: visita il nodo V se non è stato visitato;
 1. per ogni nodo adiacente $V \rightarrow adj$
 2. visita_profondità($G \rightarrow adj$)

Osservazione

- La visita in profondità è strettamente legata alla politica di gestione di uno stack (come tutti gli algoritmi ricorsivi);

Complessità Computazionale della DFS

- I cicli della *DFS*, tranne la *DFS_visit*, impiegano un tempo $\Theta(V)$.
- La *DFS_visit* viene chiamato una sola volta per ogni vertice $v \in V$. Il ciclo nella *DFS_visit* viene eseguito $|Adj[v]|$ volte.
- Poiché $\sum_{v \in V} Adj[v] = \Theta(E)$ allora il tempo totale di esecuzione è dato da $\Theta(V + E)$.

Teorema della Parentesi

- In ogni DFS di un grafo $G = (V, E)$, per ogni coppia di vertici u e v , può essere soddisfatta solo una delle tre condizioni:
 - $[d[u], f[u]]$ è contenuto in $[d[v], f[v]]$; inoltre u è un discendente di v in un *albero DF*.
 - $[d[v], f[v]]$ è contenuto in $[d[u], f[u]]$; inoltre v è un discendente di u in un *albero DF*.
 - Gli intervalli $[d[u], f[u]]$ e $[d[v], f[v]]$ sono disgiunti; inoltre u e v non sono discendenti l'uno dell'altro nella *Foresta DFS*.

Dimostrazione

- Iniziamo con il caso in cui $d[u] < d[v]$. Ci sono due sottocasi da considerare a seconda se $d[v] < f[u]$. Il primo caso si verifica quando v è stato scoperto mentre u era grigio. Quindi v è un discendente di u e la sua visita deve essere completata prima che sia finita la visita di v e pertanto $[d[u], f[u]]$ è contenuto in $[d[v], f[v]]$

(cont.)

- Nel secondo sottocaso $d[v] > f[u]$ allora i due intervalli sono disgiunti.
- Il caso $d[v] < d[u]$ è analogo al caso $d[u] < d[v]$.

Corollario

(Annidamento degli intervalli dei discendenti)

- Il vertice v è un discendente proprio del vertice u nella *foresta* DF per un grafo $G = (V, E)$, se e soltanto se $d[u] < d[v] < f[v] < f[u]$.

Teorema del Cammino Bianco

- In una *foresta DFS* di un grafo $G = (V, E)$ il vertice v é un discendente di u se e solo se al tempo $d[u]$ in cui si scopre u v e` raggiungibile da u lungo un cammino contenente esclusivamente vertici bianchi.

Dimostrazione

- Supponiamo che v sia un discendente di u . Sia w un qualsiasi vertice lungo il cammino che è tra u e v nell'albero *DFS*. Quindi w è un discendente di u e per il corollario precedente $d[u] < d[w]$ quindi w è bianco al tempo $d[u]$.
- Supponiamo che il vertice v sia raggiungibile da u lungo un cammino di vertici bianchi al tempo $d[u]$. Supponiamo che lungo il cammino tutti i vertici diventino discendenti di u .

(cont.)

- Sia w il predecessore di v , allora per il corollario $f[w] \geq f[v]$.
- Quindi v deve essere scoperto prima che sia ultimata la visita di w e perciò deve essere $d[u] < d[v] < f[w] < f[u]$. Il teorema della parentesi implica che $d[v], f[v]$ sia contenuto in $[d[u], f[u]]$ e quindi per il corollario precedente v è discendente di u .

Classificazione degli Archi

- La DFS può essere utilizzata per classificare gli archi del grafo di input $G = (V, E)$.
- Questa classificazione può essere utilizzata per raccogliere informazioni importanti su un grafo. Ad esempio **un grafo orientato è aciclico se e solo se una visita in profondità non genera archi all'indietro.**

Tipi di Archi

- Possiamo definire quattro tipi di archi in base alla foresta DFS prodotta da una visita in profondità dal grafo G :
- **Archi d' albero**
- **Archi all' indietro**
- **Archi in Avanti**
- **Archi trasversali**

Archi d' Albero

- Sono gli archi nella foresta DF. L' arco (u, v) è un arco d' albero se v viene scoperto la prima volta durante l' esplorazione di (u, v) .

Archetti all' indietro

- Gli archetti all' indietro sono quegli archetti (u, v) che collegano un vertice u ad un antenato v in un albero DF.
- I cappi, che possono presentarsi nei grafi orientati, sono considerati archetti all' indietro.

Archi in avanti

- Gli archi in Avanti sono gli archi (u, v) (diversi dagli archi d' albero) che collegano un vertice u ad un discendente v in un albero DF.

Archivi trasversali

- Gli archi trasversali sono tutti gli altri archi.
- Possono connettere i vertici nello stesso albero DF, purchè un vertice non sia antenato dell' altro, oppure possono connettere vertici di alberi DF differenti.

Colore degli Archi

- L' algoritmo DFS possiede le informazioni necessarie per classificare gli archi che incontra.
- L' idea chiave è che ogni arco (u, v) può essere classificato in base al colore del vertice v che viene raggiunto quando l' arco viene ispezionato la prima volta:
 - **WHITE**: indica un **arco d' albero**
 - **GRAY**: indica un **arco all' indietro**
 - **BLACK**: indica un **arco in avanti** o **trasversale**